PostGres (DataBase Full Notes)

--(By Mayur Tetwar)

Note :-

1. Date is string type and string comparision is allow in postgres.

2. To get unique/DISTINCT values from cloumns :- SELECT DISTINCT name FORM employee;

3. Copy data form CSV to SQL :- copy CovidProject(WHO_Region ,Country_Region ,Confirmed ,TapakGye ,Recovered)

FROM 'C:\Users\sky\DSPython\Library\PandasCovidProject.csv'

DELIMITER ',' CSV HEADER.

4. Type Cast:- CAST(x as numeric) or x::numeric

5. Need data from both tables? → JOIN

   Need to count/sum/aggregate? → GROUP BY

   Need both? → JOIN first, then GROUP BY

6. To change date stucture :- like TO_CHAR(column of date,'YYYY-MM') or DATE_PART('month', column of date) Date formate :- '2024-01-15'

7. Sequence to write Query:-

   from -> join -> where -> group by -> having -> order by -> limit

---

Database:

To get list of DB :- \l

To connect to DB :- \c DBName

create: Create database test;

Delete: drop database test;

To get all tables :- \dt

To get description of table :- \d "tableName"

1.Operations:

create: create table person(id int, name varchar(100), city varchar(100));

insert: insert into person values (101, 'Mayur', 'Ramtek');

update: update person set city='Nagpur' where id=101;

delete: delete from person where id=101;

alter: To change tables and columns properties (update used for values, this not)

:-Ex.:ALTER TABLE table_name

ADD COLUMN column_name data_type [column_constraint],

ALTER COLUMN column_name SET DEFAULT expression,

ALTER COLUMN column_name SET NOT NULL,

ALTER COLUMN column_name DROP NOT NULL

RENAME COLUMN old_column_name TO new_column_name,

ADD [CONSTRAINT constraint_name] constraint_definition,

DROP CONSTRAINT constraint_name [RESTRICT | CASCADE];

truncate: To remove mass storage

2.Constraint:

A. Primary Key: -unique key to describe row

-Only one key per table

-not null and unique in entire table

-Ex.: create table person(id primary key int,name varchar(100));

B. Not NULL: -columns with not null values

-Ex.: create table person(id int not null, name varchar(100) not null, city varchar(100));

C. Default: -Give default value if null

-Ex.:create table person(id int not null, name varchar(100) not null, acc_type varchar(50) default 'Saving');

D. Auto-Increment(Serial): -

add Serial which increment from 1 to 2^31-1 , also use for large number bigserial

-to set start value select setval(<database_name>_<column_name>_seq,<start_number>);

-Ex.: create table person(id serial primary key, name varchar(100) not null);

E. UNIQUE:- For add unique values in columns

F. Cheak:- To add boolean condition to give table or column

-Ex.: 1.CREATE TABLE orders (

order_date DATE NOT NULL,

delivery_date DATE,

CONSTRAINT chk_delivery_after_order CHECK (delivery_date >= order_date)

-- Table-level CHECK: delivery date after order date);

2.CREATE TABLE products (

stock_quantity INTEGER CHECK (stock_quantity >= 0),

-- Column-level CHECK: quantity cannot be negative

category VARCHAR(50) CHECK (category IN ('Electronics', 'Books', 'Clothing', 'Food'))

-- Column-level CHECK: specific categories);

G. Foreign key:- the key of table which take referanse from another table primary key.

:- CREATE TABLE orders (

ord_id SERIAL PRIMARY KEY,

price NUMERIC NOT NULL,

cust_id INTEGER NOT NULL,

FOREIGN KEY (cust_id) REFERENCES customers (cust_id));

:- INSERT INTO orders (ord_date, cust_id, price) VALUES ('2024-01-01', 1, 250.00);

3.Clauses: (Conditions)

Note: 1. We can use logical opretor for below conditions like and, or and not.

2. We can use Reletional opretor like (<,>,<=,>=,!=,=).

3. in and not in :- 'in' used for to cheak value is in the data or not if yes then only feth

- Ex.: select * from employee where dept in('IT','DS');

- Ex.: select * from employee where dept not in('IT','DS'); (Other then above qurey result)

4. is null.(IMP)

5. as (ke jesa || to change name of columns).

A. Where:- to fetch particular data

- Ex.: select * from employee where emp_id=2; (return all employee with emp_id 2)

B. Between:- to get data from the range

- Ex.: select * from employee where salary between 50000 and 100000;(IMP:- left value must be less then right value)

` C. Distinct:- to get unique values from columns

- Ex.,: select DISTINCT dept from employee;

D. Order By:- sorting of data

- Ex.: select * from employee ORDER BY fname;

- for asecending(default) add ASC at last of query and DESC for desending.

- Ex2.: SELECT * FROM employee ORDER BY hire_date DESC NULLS LAST;

E. Limit:- upto limit or (if greter then limit then take original) return result

-Ex.: SELECT * FROM employee ORDER BY emp_id ASC limit 3;

F. Offset:- to skip first x rows.

-Ex.: select fname,lname,salary from employee Where salary<100000 ORDER BY salary DESC limit 3 offset 3;

G. Like:- to get similar type(dekhne me) of data .

- % used for unlimited long string of any character.

- _ used for place of any character.

- Ex.: Starts with 'A': LIKE 'A%'

Ends with 'A': LIKE '%A'

Contains 'A': LIKE '%A%'

Second character is 'A': LIKE '_A%' --(skip first character)

Next all Word start with 'A': LIKE '% A%'

--(Matches records where "A" appears anywhere in the conditions field, preceded by a space(" ")(Here))

Case-insensitive contains 'john': LIKE '%john%'.

4.String Function:

• UPPER() converts the value of a field to uppercase

• LOWER() converts the value of a field to lowercase

• LENGTH() returns the length of the value in a text field

• SUBSTRING() extracts a substring from a string => substring(name,1,3) OR substring(name from 1 for 3)

• NOW() returns the current system date and time

• FORMAT() used to set the format of a field

• CONCAT() adds two or more strings together => || also use (like, 'a' || name)

• CONCAT_WS() adds two or more strings together with supreter (like, CONCAT_WS('_',fname,lname))

• REPLACE() Replaces all occurrences of a substring within a string, with a new substring

• TRIM() removes leading and trailing spaces (or other specified characters) from a string

• LEFT(str,x) => left se x length substring of str and RIGHT(str,x) => right se x char of str

• POSITION('s','str') => return index of s in str.

• SPLIT_PART('Enter String','Enter spliting part','Enter part number') => SELECT SPLIT_PART('Mayur Laxman Tetwar', ' ', -1) AS last_name;

-- Result: Tetwar

5.Aggregate function :

Aggregate function performs a calculation on multiple values and returns a single value.

• COUNT() returns number of values

• SUM() returns sum of all values

• AVG() returns average value

• MAX() returns maximum value

• MIN() returns minimum value

• ROUND() Rounds a number to a specified number of decimal places. => amount(salary,2) upto 2 decimal

• STRING_AGG(expression,separator) return concated string of all values comes under this with separator as mention.

6.1.GROUP BY :

The GROUP BY statement group rows that have the same values into summary rows.

It is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns

• Example.: SELECT mode, SUM(amount) AS total

FROM payment

GROUP BY mode;

6.2.Group By Rollup :

Ex.:- select coalesce(p_name,'Total'),--(coalesce function used to chnage null values to another values (here if p_name is null them return 'Total'))

sum(total_price)

from billing_info GROUP BY rollup(p_name)

order by sum(total_price);

7.HAVING clause:

The HAVING clause is used to apply a filter on the result of GROUP BY based on the specified condition.

The WHERE clause places conditions on the selected columns, whereas the HAVING clause places conditions on groups created by the GROUP BY clause

- Example.: SELECT mode, COUNT(amount) AS total

  FROM payment GROUP BY mode

  HAVING COUNT(amount) >= 3

  ORDER BY total DESC;

8.JOIN:

JOIN operation is used to combine rows from two or more tables based on a related column between them.

TYPES:

1.CROSSE JOIN:- Every row from one table is combined with every row from another table.(NOT IMP)

2.INNER JOIN:- note:- you can use INNER JOIN or JOIN.Returns only the rows where there is a match between the specified columns in both. the left (or first) and right (or second) tables.(like union(similar part) of two tables)

Ex:- 1. select * from customers c INNER JOIN orders o ON c.cust_id=o.cust_id;

2.select c.cust_name,sum(o.price) as Total,count(c.cust_id) from customers c INNER JOIN orders on c.cust_id=o.cust_id group by c.cust_name;

3.LEFT JOIN:- Returns all rows from the left (or first) table and the matching rows from the right (or second) table.

Ex:- 1.select * from customers c LEFT JOIN orders o ON c.cust_id=o.cust_id;

2.select c.cust_name,sum(o.price) from customers c LEFT JOIN orders o

ON c.cust_id=o.cust_id GROUP BY c.cust_name having sum(o.price)!=0;

4.RIGHT JOIN:- Returns all rows from the right (or second) table and the matching rows from the left (or first) table.

5.Full OUTER JOIN:- Returns all records when there is a match in either left or right table. (return mathching rows and an mathchhing with null values).

Ex.: SELECT * FROM customers c FULL OUTER JOIN orders o ON o.cust_id = c.cust_id;

6.SELF JOIN:- SELF Joins are powerful for comparing values in a column of rows with the same table.

Ex.:- select * from emp e1 JOIN emp e2 ON e2.emp_id = e1.manager_id;

MANY-TO-MANY Relation:-

create 2 tables have there unique Primary Key , Make another table which contians mapping of this two PK.

1. Ex:-

CREATE TABLE courses (c_id SERIAL PRIMARY KEY,name VARCHAR(100) NOT NULL);

CREATE TABLE students (s_id SERIAL PRIMARY KEY,name VARCHAR(100) NOT NULL);

CREATE TABLE enrollment (enrollment_id SERIAL PRIMARY KEY,s_id INT NOT NULL,c_id INT NOT NULL,enrollment_date DATE NOT NULL,

FOREIGN KEY (s_id) REFERENCES students(s_id),

FOREIGN KEY (c_id) REFERENCES courses(c_id));

select s.name,c.name,EXTRACT(YEAR FROM e.enrollment_date) from enrollment e

JOIN students s ON s.s_id = e.s_id

JOIN courses c ON c.c_id = e.c_id;

2. Ex:- (to merge more then 2 tables)

select c.cust_name,o.ord_id,o.ord_date,p.p_name,oi.quantity,p.price,

(oi.quantity*p.price) as total_price

from order_items oi

JOIN orders o ON o.ord_id=oi.ord_id

JOIN products p ON p.p_id=oi.p_id

JOIN customers c ON o.cust_id=c.cust_id;

9.1. UNION:-

The SQL UNION clause/operator is used to combine/concatenate the results of two or more SELECT statements without returning any duplicate rows and keeps unique records.

Ex.:- SELECT cust_name, cust_amount from custA

UNION

SELECT cust_name, cust_amount from custB.

9.2. UNION ALL:- In UNION ALL everything is same as UNION, it combines/concatenate two or more table but keeps all records, including duplicates.

10.SubQuary:

A Subquery or Inner query or a Nested query allows us to create complex query on the output of another query.

Ex.:- select * from employee WHERE salary=(select max(salary) from employee);

11.1.Case Statement :-

• The CASE expression goes through conditions and returns a value when the first condition is met (like if-then-else statement). If no conditions are true, it returns the value in the ELSE clause.

• If there is no ELSE part and no conditions are true, it returns NULL.(IMP)

Ex.:- SELECT CASE

WHEN salary>50000 THEN 'High'

WHEN salary BETWEEN 48000 AND 55000 THEN 'Mid'

ELSE 'Low'

END AS sal_cat,count(emp_id) FROM employee GROUP BY sal_cat;

11.2.Case Expression:- SELECT customer_id,

CASE amount--(expresion)

WHEN 500 THEN 'Prime Customer'--(if amount==500)

WHEN 100 THEN 'Plus Customer'--(if amount==100)

ELSE 'Regular Customer'--(else)

END AS CustomerStatus

FROM payment;

12.Views :-

To sotre the bigger query into small one.

Ex.:- create view custom_info as

select name,paid_amount from customers;

--(In above statement , one view is created named as custom_info . The result is saved in this tempary table)

--(You can see this using select * from custom_info;)

13.STORED Routine:-

An SQL statement or a set of SQL Statement that can be stored on database server which can be call no. of times. (To store SQL query in procedure or functions).

Types:-

1.STORED Procedure

Set of SQL statements and procedural logic that can perform operations such as inserting, updating, deleting, and querying data.

(Genraly for CRUD opretions).

Ex.:-

```
CREATE OR REPLACE PROCEDURE update_emp_salary( p_employee_id INT,
p_new_salary NUMERIC)

LANGUAGE plpgsql AS $$

BEGIN

UPDATE employees SET salary = p_new_salary WHERE emp_id =
p_employee_id;

END;$$;

--- Call:- CALL update_emp_salary(3,100000);
```

2.USER DEFINED FUNCTIONS

custom function created by the user to perform specific operations and return a value.

Ex.:-

```
CREATE OR REPLACE FUNCTION dept_max_sal_emp(dept_name VARCHAR)

RETURNS TABLE(emp_id INT, fname VARCHAR, salary NUMERIC)  AS $$

BEGIN

RETURN QUERY

select fname,lname,salary from employee

where dept=dept_name and salary=(

select max(salary) from employee

where dept=dept_name);
```

END;

$$ LANGUAGE plpgsql;

--- Call:- select * from dept_max_sal_emp('DS'); -> return tables of employee which have highest salary in 'DS' dept (HERE).

---

14. Window Function:

this applies aggregate, ranking and analytic functions over a particular window (set of rows). And 'OVER' clause is used with window functions to define that window.

Syntax:- SELECT column_name(s), fun() OVER ([<PARTITION BY Clause>]

[<ORDER BY Clause>] [<ROW or RANGE Clause>])

1. Select a function :- • Aggregate functions

• Ranking functions

• Analytic functions

2. Define a Window :-

• PARTITION BY -> asign same value(calculated) to all partition row (similar to group by but not do grouping)

• ORDER BY -> used to calculate running values in perticular order (First order by then sum from first row)

• ROWS -> range from cureent row to upper ans lower rows (ROWS BETWEEN UNBOUNDED(n)(uper ke rows ka number) PRECEDING AND UNBOUNDED(n)(niche wale rows) FOLLOWING)

3. Ranking :-

• ROW_NUMBER() :- Direct counting of rows from up to down.

• RANK() :- Give rank to each value if repeted then same and skip the next value. Ex.:- 1,2,2,4,5,6,6,8

• DENSE_RANK() :- Give rank to ach value if repeted then same without skip

Ex.:- 1,2,2,3,4,5,5,6

- PERCENT_RANK() :- Give rank percentage which express that how many percentage of rows
- NTILE(n) :- divides rows into n buckets

4. Analytic functions:-

- FIRST_VALUE(column):- first value in window frame
- LAST_VALUE(column):- last value in window frame
- LEAD(column,offset):- This return previous value of this column else NULL
- LAG(column,offset):- This return next value of this column else NULL

Ex.:- 1. Aggregate functions:-

```
select emp_id,dept, count(emp_id) over (PARTITION BY dept order by emp_id) as "emp_count" from employee;
```

2. Ranking functions:-

```
select fname,dense_rank() over(order by salary desc) from employee;
```

3. Analytic functions:-

```
select dept,
first_value(dept) over(order by dept) as "FIRST_VALUE",
last_value(dept) over(order by dept) as "LAST_VALUE",
lead(dept,2) over(order by dept) as "LEAD",
lag(dept,2) over(order by dept) as "LAG"
from employee;
```

---

15. CTE:- CTE (Common Table Expression) is a temporary result set that you can define within a query to simplify complex SQL statements

In journal cases, cte is used with joins to solve the problem.

Ex.:-

```
with max_sal as (
        select dept,max(salary) as max_salary from employee group by dept
)
select e.fname,e.dept,e.salary
from employee e join max_sal m
on e.dept=m.dept where e.salary=m.max_salary;
```

---

16. Trigger :- Call the function when some thing is occur like (insert, update or delete) .
